

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-a2770eaacc46ca81a.jsonl`
- SHA-256: `4d9c851406b87de3f20fe00ef9c53907154667741c5b74ef81a11c3279e0aeeb`
- Source modified: `2026-06-22T15:07:18+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T15:05:03.849Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: dispatch-correctness

Severity: minor

Title: Cloud pre-dispatch rejections set segmentation_ran=True, corrupting the observability report (fake 30fps 'fallback')

File: backend/main.py:607-610

Detail: `if remote is not None: segmentation\_ran = True` is set whenever the cloud path returns ANY dict, including the pre-dispatch rejections produced by `_maybe_dispatch_remote._failed` (unresolvable input, local-only input, missing storage.output_root, dispatch error) ? cases where NO segmentation ran at all. The `emit_indexer_report` in the finally then receives `segmentation_ran=True`, and `backend/reporting/harvest.py:209-212` uses that to mislabel `fps_source` as 'fallback' and inject an assumed `fps=30.0` for a video that was never segmented. The in-process equivalents of these rejections (validation failure / unknown segmenter) leave `segmentation_ran=False`, so this is an asymmetry that makes the per-video report lie for cloud config/input errors.

Fix: Only mark `segmentation_ran=True` when the cloud job actually executed (e.g. `remote.get('status') == 'success'`), or return a sentinel from `_maybe_dispatch_remote` distinguishing 'job ran' from 'pre-dispatch rejection' and gate `segmentation_ran` on it.

2. assistant (2026-06-22T15:05:07.548Z)

I'll verify this finding by examining the cited code in the bhi-cloudrun repository.

3. user (2026-06-22T15:05:08.770Z)

```
backend/main.py | 100 +++++
tests/test_api_cloud_dispatch.py | 212 +++++
2 files changed, 312 insertions(+)
```

4. assistant (2026-06-22T15:05:10.900Z)

Let me look at the actual diff for the cited code region.

5. user (2026-06-22T15:05:12.263Z)

```
diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -1,10 +1,12 @@
```

```

import os
import sys
import argparse
+import csv
import json
import logging
import shutil
import socket
+import tempfile
import threading
import uuid
import webbrowser

@@ -436,6 +438,94 @@ def download_reel(filename: str):
    return FileResponse(path, media_type=media, filename=name)

+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
(cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                   "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                             play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
+    == "cloud_run``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
+    from backend.compute import ComputeJobSpec, get_compute_backend
+
+    compute = get_compute_backend(global_config)

```

```

+   if compute is None:
+       return None # default-OFF ? run the in-process segmenter below (unchanged)
+
+   storage_cfg = getattr(global_config, "storage", None)
+
+   def _failed(msg: str) -> Dict[str, Any]:
+       # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+       db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+       return {"video_id": video_id, "status": "failed", "message": msg,
+               "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+   # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+   try:
+       input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+   except ValueError as e:
+       return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+   if input_ref.backend == "local":
+       return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+                       "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+   out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+   if not out_root:
+       return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+                       "? that's where the Cloud Run job writes the result.")
+
+   notify("dispatching (cloud_run)")
+   logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+   spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+   try:
+       result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
+   except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+       logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+       return _failed(f"cloud dispatch error: {e}")
+   if result.returncode != 0:
+       return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+   notify("ingesting (cloud_run)")
+   storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+   _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+   # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+   segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+   _finalize_reingest(video_id, segments, play_wm, cand_wm)
+   return {"video_id": video_id, "status": "success",
+           "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+           "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+   def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
+       """The full hash ? validate ? segment ? report pipeline for one video.

@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
+       # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
+       # if the run yields segments; otherwise drop the new partial rows and keep the old.
+       play_wm, cand_wm = db_instance.segment_watermarks(video_id)

```

```

+         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
+         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+         # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+         # they're dropped on the cloud path in this slice ? a documented follow-up.)
+         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+         if remote is not None:
+             segmentation_ran = True
+             response = remote
+             return response
        segmenter = segmenter_cls(db_instance, global_config)
        try:
            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)

```

6. assistant (2026-06-22T15:05:15.249Z)

Now I have the key code. Let me see the full context around lines 607-610 and the `segmentation\_ran` / `emit\_indexer\_report` flow in the `finally` block.

7. user (2026-06-22T15:05:16.360Z)

```

304252b feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 -> API), default-
OFF
df3b225 docs(nightly): one-stop operations manual for the nightly engine-regression (#325)
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
01/02 (#323)
32f4ee6 test(eval): re-ground served Gate-A litmus + QUALITY_ITERATIONS on the 15-clip golden
corpus (#322)
db74f9b docs(data): mark DATA_IN_GCS $7b blocker #1 resolved by #319 (#321)

```

8. user (2026-06-22T15:05:20.802Z)

```

560         seg_name = req.segmenter or default_seg
561         segmenter_actual = None
562         segmentation_ran = False
563         response: Optional[Dict[str, Any]] = None
564         try:
565             if failures:
566                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567                 # status; it no longer destroys the video's prior good segments.
568                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569                 response = {
570                     "video_id": video_id,
571                     "status": "failed",
572                     "message": "Video failed validation checks.",
573                     "results": [r.model_dump() for r in val_results],
574                 }
575                 return response
576
577             # 2. Run segmenter & indexer
578             logger.info("[API] Video passed checks. Segmenting and indexing...")
579             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
580
581             segmenter_cls = segmenter_registry.get(seg_name)
582             if not segmenter_cls:
583                 # Submit-time validation already 400s unknown names; this is the defensive

```

```

584         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
585         available = list(segmenter_registry.list_available().keys())
586         response = {
587             "video_id": video_id,
588             "status": "failed",
589             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
590             "results": [r.model_dump() for r in val_results],
591             "play_segments": [],
592         }
593         return response
594
595         logger.info(f"[API] Using segmenter: {seg_name}")
596         notify(f"segmenting ({seg_name})")
597         segmenter_actual = seg_name
598         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
599         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
600         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605         # they're dropped on the cloud path in this slice ? a documented follow-up.)
606         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607         if remote is not None:
608             segmentation_ran = True
609             response = remote
610             return response
611         segmenter = segmenter_cls(db_instance, global_config)
612         try:
613             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
614         except Exception:
615             # A raising segmenter must not leave half-written rows: restore the pre-run
616             # state (same destroy-after-confirm contract as the Failure branch), then
let
617             # the job worker record the failure.
618             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
619             raise
620         segmentation_ran = True
621
622         if isinstance(result, Failure):
623             logger.error(f"[API] Segmenter failed: {result.message}")
624             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
625             response = {
626                 "video_id": video_id,
627                 "status": "failed",
628                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
629                 "results": [r.model_dump() for r in val_results],
630                 "play_segments": [],
631             }
632             return response
633
634         segments = result.value if hasattr(result, "value") else result
635         notify("finalizing")
636         _finalize_reingest(video_id, segments, play_wm, cand_wm)
637

```

```

638         response = {
639             "video_id": video_id,
640             "status": "success",
641             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
642             "results": [r.model_dump() for r in val_results],
643             "play_segments": segments,
644         }
645         return response
646     finally:
647         # Always emit the per-video observability report (next to the video, or to
648         # report.output_dir). Never raises. Surface the paths in the response.
649         paths = emit_indexer_report(
650             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
651             val_results=val_results, val_context=val_context,
652             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
653             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
654         )
655         if paths and isinstance(response, dict):
656             response["reports"] = paths
657
658
659     @app.post("/api/process", status_code=202)
660     def process_video(req: ProcessRequest, request: Request):
661         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
662
663         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
664         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
665         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
666         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
667         """
668         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
669         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
670         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
671         try:
672             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
673         except edge_auth.EdgeAuthError as e:
674             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
675
676         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
677         try:
678             validate_input_path(req.video_path, allowed_root=allowed_root)
679             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported

```

9. assistant (2026-06-22T15:05:23.292Z)

Now I have a clear picture of the structure. The finding hinges on what `segmentation\_ran` does in `emit\_indexer\_report` / `harvest.py`. Let me examine the harvest reporting logic.

10. user (2026-06-22T15:05:24.370Z)

```

180     video_path: Optional[str],
181     config: Dict[str, Any],
182     val_results: List[Any],
183     val_context: Optional[Dict[str, Any]],
184     play_segments: List[Dict[str, Any]],
185     candidates: List[Dict[str, Any]],
186     segmenter_requested: Optional[str] = None,
187     segmenter_actual: Optional[str] = None,

```

```

188     was_reingest: bool = False,
189     segmentation_ran: bool = True,
190     tool_version: Optional[str] = None,
191     spec: Optional["obsreport.ReportSpec"] = None,
192     run_signals: Optional[Dict[str, Any]] = None,
193     **recorder_kwargs: Any,
194 ) -> "obsreport.RunRecorder":
195     """Assemble a finalized recorder for an indexer run (does NOT write)."""
196     rec = obsreport.RunRecorder(
197         "badminton-highlight-indexer",
198         tool_version=tool_version,
199         video_id=video_id,
200         video_id_kind="file_md5",
201         video_path=video_path,
202         spec=spec if spec is not None else load_spec(),
203         **recorder_kwargs,
204     )
205     vm = video_meta_from_context(val_context, video_path)
206     # If we never probed an fps but segmentation still ran, the segmenter must have
207     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
208     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?
209     # (fallback)".
210     if vm.get("fps_source") == "unknown" and segmentation_ran:
211         vm["fps_source"] = "fallback"
212         if vm.get("fps") is None:
213             vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
214                             / 1280)
215     rec.set_video_meta(**vm)
216     _validation_stage(rec, val_results)
217     config_default = (config.get("indexing") or {}).get("default_segmenter")
218     _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
219                         segmenter_actual, config_default, was_reingest,
220                         segmentation_ran)
221     _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran,
222              run_signals)
223     _highlights(rec, play_segments, vm)
224     rec.finalize()
225     return rec

```

11. assistant (2026-06-22T15:05:28.156Z)

This confirms the harvest.py behavior the finding describes: when `fps\_source == "unknown"` and `segmentation\_ran` is True, it sets `fps\_source = "fallback"` and injects `fps = 30.0`. Now the critical question is whether the pre-dispatch rejection paths actually reach this with `fps\_source == "unknown"`. Let me trace the validation context and whether fps gets probed for the cloud rejection cases. Let me look at the test file to understand the intended behavior and what scenarios are covered.

12. user (2026-06-22T15:05:28.836Z)

```

1     """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3     Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
4     the rally
5     intervals back into the API DB, so /api/query + /api/compile (local stitch) are
6     unchanged ? all
7     GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
8     intervals.csv + the
9     done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
10    compute_target=cloud_run
11    dispatches; every other target runs the in-process segmenter byte-identically.
12    """

```

```

9     import csv
10    import json
11    import os
12
13    import pytest
14
15    pytest.importorskip("httpx")
16    from fastapi.testclient import TestClient
17
18    import backend.compute as compute_pkg
19    import backend.main as m
20    from backend.compute.cloud_run import CloudRunJobClient
21    from backend.config.models import AppConfig
22    from backend.infrastructure.database import Database
23    from backend.infrastructure.execution_policy import ExecutionPolicy
24    from backend.pipeline.validators.base import ValidationResult
25
26    _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
27
28
29    def _pass():
30        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
31
32
33    class _InlineExecutor:
34        def submit(self, fn, *a, **k):
35            fn(*a, **k)
36
37
38    class _FakeCloudClient(CloudRunJobClient):
39        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
40        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
41
42        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
43            self.calls: list[list[str]] = []
44            self.video_id, self.returncode = video_id, returncode
45            self.rows = rows if rows is not None else [(2.74, 6.91, 5, "winner"),
46                                                         (8.74, 11.38, 3, "unforced_error")]
47
48        def run_job(self, job_name, argv, *, region, project=None):
49            self.calls.append(list(argv))
50            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
51            done_uri = argv[argv.index("--done-uri") + 1]
52            if self.returncode == 0:
53                outputs = os.path.join(out_uri, "outputs", self.video_id)
54                os.makedirs(outputs, exist_ok=True)
55                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
56                    json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
57                              "status": "success"}, f)
58                with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
59                    w = csv.writer(f)
60                    w.writerow(["start", "end", "shot_count", "ending_reason"])
61                    for s, e, sc, er in self.rows:
62                        w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
63                os.makedirs(os.path.dirname(done_uri), exist_ok=True)
64                with open(done_uri, "w", encoding="utf-8") as f:
65                    json.dump({"video_id": self.video_id, "returncode": self.returncode,
66                              "segment_count": len(self.rows), "status": "success"}, f)
67            return "exec-1"
68
69
70    @pytest.fixture

```



```

71     def cloud_env(tmp_path, monkeypatch):
72         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
73         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
74         db = Database(str(tmp_path / "t.db"))
75         bucket = tmp_path / "bucket"
76         bucket.mkdir()
77         cfg = AppConfig.model_validate({
78             "deployment": {"compute_target": "cloud_run"},
79             "storage": {"backend": "local", "output_root": str(bucket)},
80         })
81         monkeypatch.setattr(m, "db_instance", db)
82         monkeypatch.setattr(m, "global_config", cfg)
83         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
84         # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
85         fake_local = tmp_path / "fetched.mp4"
86         fake_local.write_bytes(b"FAKEVIDEOBYTES")
87         monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg:
str(fake_local))
88         monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
89         return db, bucket, monkeypatch
90
91
92     def _inject_fake(monkeypatch, fake):
93         """Make backend.compute.get_compute_backend (re-imported inside
_maybe_dispatch_remote) build a
94         real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token."""
95         real = compute_pkg.get_compute_backend
96         monkeypatch.setattr(compute_pkg, "get_compute_backend",
97                             lambda config: real(config, run_client=fake, policy=_FAST,
token="tok"))
98
99
100     def _meta(row):
101         md = row.get("metadata")
102         return json.loads(md) if isinstance(md, str) else (md or {})
103
104
105     def test_cloud_dispatch_ingests_segments(cloud_env):
106         db, _bucket, monkeypatch = cloud_env
107         fake = _FakeCloudClient()
108         _inject_fake(monkeypatch, fake)
109
110         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
111
112         assert out["status"] == "success"
113         assert len(out["play_segments"]) == 2
114         rows = db.query_play_segments(out["video_id"], {})
115         assert len(rows) == 2
116         metas = [_meta(r) for r in rows]
117         assert all(md["source"] == "cloud_run" for md in metas)
118         assert {md.get("shot_count") for md in metas} == {5, 3}
119         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
120
121
122     def test_cloud_dispatch_builds_correct_spec(cloud_env):
123         _db, bucket, monkeypatch = cloud_env
124         fake = _FakeCloudClient()
125         _inject_fake(monkeypatch, fake)
126

```

```

127         m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
128
129         assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
130         argv = fake.calls[0]
131         assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
132         assert argv[argv.index("--out-uri") + 1] == str(bucket)
133         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
134         joined = " ".join(argv)
135         assert "deployment.compute_target=local_gpu" in joined # container runs INLINE
(never re-dispatch)
136         assert "indexing.detector_impl=native" in joined
137
138
139     def test_cloud_failure_marks_failed_no_rows(cloud_env):
140         db, _bucket, monkeypatch = cloud_env
141         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
142         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
143         fake = _FakeCloudClient(returncode=1)
144         _inject_fake(monkeypatch, fake)
145
146         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
147         assert out["status"] == "failed"
148         assert "returncode 1" in out["message"]
149         assert out["play_segments"] == []
150         assert db.query_play_segments(out["video_id"], {}) == []
151
152
153     def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
154         _db, _bucket, monkeypatch = cloud_env
155         fake = _FakeCloudClient()
156         _inject_fake(monkeypatch, fake)
157         local_path = str(tmp_path / "local_only.mp4")
158
159         out = m._execute_processing(m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid"))
160         assert out["status"] == "failed"
161         assert "cloud input" in out["message"]
162         assert fake.calls == [] # never dispatched a local-only path
163
164
165     def test_default_off_runs_in_process(tmp_path, monkeypatch):
166         """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
167         byte-identically (the gate is inert). No Cloud Run dispatch."""
168         db = Database(str(tmp_path / "t.db"))
169         cfg = AppConfig() # compute_target defaults to ""
170         monkeypatch.setattr(m, "db_instance", db)
171         monkeypatch.setattr(m, "global_config", cfg)
172         monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
173         video = tmp_path / "v.mp4"
174         video.write_bytes(b"hashable-bytes")
175
176         class _InProcSeg:
177             def __init__(self, db_, cfg_):
178                 self.db = db_
179
180             def process_video(self, vid, path, *a, **k):
181                 sid = self.db.add_play_segment(vid, 0.0, 5.0)
182                 return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
183

```

```

184 monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
185 out = m._execute_processing(m.ProcessRequest(video_path=str(video),
segmenter="motion"))
186 assert out["status"] == "success"
187 assert len(out["play_segments"]) == 1
188 assert "cloud_run" not in out["message"]
189
190
191 def test_full_api_process_to_query_via_cloud(cloud_env):
192     """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
193     to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
194     db, _bucket, monkeypatch = cloud_env
195     fake = _FakeCloudClient(video_id="vidCloud2")
196     _inject_fake(monkeypatch, fake)
197     client = TestClient(m.app)
198
199     r = client.post("/api/process", json={"video_path": "gs://b/clip.mp4", "segmenter":
"wasb_hybrid"})
200     assert r.status_code == 202
201     job_id = r.json()["job_id"]
202
203     job = client.get(f"/api/jobs/{job_id}").json()
204     assert job["status"] == "done"
205     video_id = job["video_id"]
206     assert video_id
207
208     q = client.post("/api/query", json={"video_id": video_id})
209     assert q.status_code == 200
210     segs = q.json()["play_segments"]
211     assert len(segs) == 2
212     assert all(_meta(s)["source"] == "cloud_run" for s in segs)
213

```

13. assistant (2026-06-22T15:05:32.950Z)

Now let me understand the validation context (`val\_context`) and how `fps\_source` gets populated. The finding claims pre-dispatch rejections reach the report with `fps\_source == "unknown"`. Let me check `video\_meta\_from\_context` and how `val\_context` is set in `\_execute\_processing`.

14. user (2026-06-22T15:05:34.358Z)

```

backend/main.py:555: val_results, val_context = registry.run_all_with_context(local_video,
global_config)
backend/main.py:651: val_results=val_results, val_context=val_context,
backend/main.py:1118: val_results, val_context =
registry.run_all_with_context(local_video, global_config)
backend/main.py:1167: val_results=val_results, val_context=val_context,
backend/reporting/harvest.py:31: def video_meta_from_context(val_context: Optional[Dict[str,
Any]], video_path: Optional[str]) -> Dict[str, Any]:
backend/reporting/harvest.py:32: """Derive VideoMeta fields from the validators' shared
ffprobe_meta. fps_source
backend/reporting/harvest.py:34: meta: Dict[str, Any] = {"fps_source": "unknown"}
backend/reporting/harvest.py:41: ffmeta = (val_context or {}).get("ffprobe_meta") or {}
backend/reporting/harvest.py:55: meta["fps_source"] = "probed"
backend/reporting/harvest.py:183: val_context: Optional[Dict[str, Any]],
backend/reporting/harvest.py:205: vm = video_meta_from_context(val_context, video_path)
backend/reporting/harvest.py:209: if vm.get("fps_source") == "unknown" and segmentation_ran:
backend/reporting/harvest.py:210:     vm["fps_source"] = "fallback"

```

15. user (2026-06-22T15:05:37.191Z)

```

31 def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:

```

```

Optional[str]) -> Dict[str, Any]:
32     """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33     distinguishes a real probe from the absence of one (the fallback footgun)."""
34     meta: Dict[str, Any] = {"fps_source": "unknown"}
35     if video_path and os.path.exists(video_path):
36         try:
37             meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38         except OSError:
39             pass
40
41     ffmata = (val_context or {}).get("ffprobe_meta") or {}
42     streams = ffmata.get("streams") or []
43     fmt = ffmata.get("format") or {}
44     if streams:
45         s = streams[0]
46         w, h = s.get("width"), s.get("height")
47         meta["width"], meta["height"] = w, h
48         if w and h:
49             meta["resolution"] = f"{w}x{h}"
50         fps_str = s.get("r_frame_rate", "0/1")
51         try:
52             num, den = (int(x) for x in fps_str.split("/"))
53             if den:
54                 meta["fps"] = round(num / den, 2)
55                 meta["fps_source"] = "probed"
56         except (ValueError, ZeroDivisionError):
57             pass
58         if s.get("codec_name"):
59             meta["container"] = fmt.get("format_name") or s.get("codec_name")
60     if fmt.get("duration"):

```

16. assistant (2026-06-22T15:05:42.174Z)

Now let me check what `val\_context` actually contains in the cloud path. In `\_execute\_processing`, `val\_results`, `val\_context` come from `registry.run\_all\_with\_context(local\_video, ...)` at line 555. So validators DO run (and probe fps) before the dispatch. The key question: does `ffprobe\_meta` get populated for the cloud rejection cases?

In the cloud path, `local\_video` is fetched via `acquire\_local\_input` (a real local file for hashing/validation), so validators run against an actual local file and `ffprobe\_meta` would typically be populated? `fps\_source == "probed"`. Let me check line 540-560 to confirm `local\_video` is always a real local file before validation.

17. user (2026-06-22T15:05:43.254Z)

```

525         "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
526         '{seg_name}').",
527         "results": [r.model_dump() for r in val_results], "play_segments": segments}
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.
531
532         This is the former synchronous /api/process body, unchanged in behavior, now run on
533         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
534         the sync endpoint used to return (UI compatibility); the per-video observability
535         report is still emitted in `finally:` and grafted as response["reports"].
536
537         `progress` is an optional callable(stage: str) used to surface coarse job progress.
538         Raises on unexpected internal errors ? the caller records those as a job failure
539         (after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542

```

```

543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
545         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]
557
558         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
559         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560         seg_name = req.segmenter or default_seg
561         segmenter_actual = None
562         segmentation_ran = False
563         response: Optional[Dict[str, Any]] = None
564         try:

```

18. assistant (2026-06-22T15:05:49.438Z)

Now I have the full picture. Critical reasoning:

For the cloud path to be reached at all (``_maybe_dispatch_remote`` returning a dict), the code must pass validation (``failures`` empty, otherwise it returns early at line 575) and have a valid local file (since ``acquire_local_input`` already fetched it for hashing+validation). So by the time ``_maybe_dispatch_remote`` runs, validators have run against a real local file and ``ffprobe_meta`` should be populated? ``fps_source == "probed"`` in the normal case.

This means for the pre-dispatch rejection cases, ``fps_source`` would typically already be ``"probed"`` (from a real ffprobe of the fetched local file), NOT ``"unknown"``. The fps fabrication at `harvest.py:209-212` only triggers when ``fps_source == "unknown"``. Let me verify whether ffprobe could fail/be absent even when validators pass, which would leave ``fps_source == "unknown"``.

19. user (2026-06-22T15:05:50.977Z)

```

backend/pipeline/validators/base.py:42: def run_all_with_context(self, video_path: str,
config: Dict[str, Any]):
backend/pipeline/validators/base.py:46:         ``context['ffprobe_meta']`` produced by
FormatValidator). ``run_all``
backend/pipeline/validators/base.py:66:         results, _ =
self.run_all_with_context(video_path, config)
backend/pipeline/validators/format.py:47:         context["ffprobe_meta"] = info
backend/pipeline/validators/resolution_fps.py:14:         meta = context.get("ffprobe_meta")
Binary file backend/pipeline/validators/__pycache__/base.cpython-313.pyc matches
Binary file backend/pipeline/validators/__pycache__/format.cpython-313.pyc matches
Binary file backend/pipeline/validators/__pycache__/resolution_fps.cpython-313.pyc matches

```

20. user (2026-06-22T15:05:54.595Z)

```

42         def run_all_with_context(self, video_path: str, config: Dict[str, Any]):
43             """Runs all validators and returns ``(results, context)``.
44

```

```

45         The per-run ``context`` accumulates shared values (notably
46         ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
47         discards it; callers that want media metadata for an observability report
48         use this variant instead of re-probing.
49         """
50         results = []
51         context: Dict[str, Any] = {} # Shared context (e.g. for caching ffprobe
outputs)
52         for val in self._validators:
53             try:
54                 res = val.validate(video_path, config, context)
55                 results.append(res)
56             except Exception as e:
57                 results.append(ValidationResult(
58                     validator_name=val.name,
59                     passed=False,
60                     message=f"Validator threw an unexpected error: {str(e)}"
61                 ))
62         return results, context
63
64     def run_all(self, video_path: str, config: Dict[str, Any]) ->
List[ValidationResult]:
65         """Runs all registered validators in sequence, accumulating results."""
66         results, _ = self.run_all_with_context(video_path, config)

```

21. user (2026-06-22T15:05:57.234Z)

```

1     import subprocess
2     import json
3     import os
4     from backend.pipeline.validators.base import VideoValidator, ValidationResult
5     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
6     from typing import Dict, Any
7
8     class FormatValidator(VideoValidator):
9         @property
10         def name(self) -> str:
11             return "format_check"
12
13         @property
14         def description(self) -> str:
15             return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
16
17         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
18             if not os.path.exists(video_path):
19                 return ValidationResult(
20                     validator_name=self.name,
21                     passed=False,
22                     message=f"File not found: {video_path}"
23                 )
24
25             cmd = [
26                 ffprobe_bin(),
27                 "-v", "error",
28                 "-select_streams", "v:0",
29                 "-show_format",
30                 "-show_streams",
31                 "-of", "json",
32                 video_path
33             ]
34
35             try:
36                 result = subprocess.run(cmd, capture_output=True, text=True, check=True)

```

```

37         info = json.loads(result.stdout)
38
39         if not info.get("streams") or not info.get("format"):
40             return ValidationResult(
41                 validator_name=self.name,
42                 passed=False,
43                 message="Failed to retrieve valid video streams from the file."
44             )
45
46         # Store the metadata in context for subsequent validators to utilize
47         context["ffprobe_meta"] = info
48
49         stream = info["streams"][0]
50         fmt = info["format"]
51
52         details = {
53             "format_name": fmt.get("format_name"),
54             "duration": float(fmt.get("duration", 0)),
55             "size_bytes": int(fmt.get("size", 0)),
56             "codec_name": stream.get("codec_name"),
57         }
58
59         # Check if container name includes mp4
60         if "mp4" not in fmt.get("format_name", "").lower():
61             return ValidationResult(
62                 validator_name=self.name,
63                 passed=False,
64                 message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
65                 details=details
66             )
67
68         return ValidationResult(
69             validator_name=self.name,
70             passed=True,
71             message="Video format is valid MP4 container.",
72             details=details
73         )
74
75     except subprocess.CalledProcessError as e:
76         return ValidationResult(
77             validator_name=self.name,
78             passed=False,
79             message="Video stream is corrupted or unreadable by ffprobe.",
80             details={"error": e.stderr}
81         )
82     except Exception as e:
83         return ValidationResult(
84             validator_name=self.name,
85             passed=False,
86             message=f"Unexpected error running ffprobe: {str(e)}"
87         )
88

```

22. assistant (2026-06-22T15:06:07.324Z)

This is decisive. `FormatValidator`` sets `context["ffprobe_meta"] = info`` (line 47) ONLY on the success path. If ffprobe succeeds and finds an MP4 with streams, the validator passes AND `ffprobe_meta`` is set ? harvest's `video_meta_from_context`` derives `fps_source == "probed"``.

If ffprobe fails for any reason (corrupted, not MP4, file not found, ffprobe error), `ffprobe_meta`` is NOT set, BUT the validator returns `passed=False`` ? `failures`` is non-empty ? `_execute_processing`` returns at line 575 BEFORE ever reaching `_maybe_dispatch_remote``.

So the coupling is tight: **`ffprobe_meta`` is set if-and-only-if validation passes** (at least

the FormatValidator passes, which is the one that populates fps). For `\_maybe\_dispatch\_remote` to even be called, all validators must pass (`failures` empty), which requires FormatValidator to pass, which requires `ffprobe\_meta` to be set, which means `fps\_source == "probed"`, not `"unknown"`.

Therefore the fabrication branch at harvest.py:209 (`fps\_source == "unknown" and segmentation\_ran`) cannot trigger for the cloud pre-dispatch rejection cases ? `fps\_source` is already `"probed"`. The finding's claimed corruption (fake 30fps fallback) requires `fps\_source == "unknown"`, which is unreachable on the cloud path that has passed validation.

Let me double-check the resolution_fps validator to be sure it doesn't fail-pass in a way that leaves `ffprobe\_meta` set but the whole thing failing ? though that wouldn't matter since failing means early-return. Let me also confirm there's no alternate validator set where FormatValidator is absent.

23. user (2026-06-22T15:06:08.039Z)

```
1      from backend.pipeline.validators.base import VideoValidator, ValidationResult
2      from typing import Dict, Any
3
4      class ResolutionFPSValidator(VideoValidator):
5          @property
6          def name(self) -> str:
7              return "resolution_fps_check"
8
9          @property
10         def description(self) -> str:
11             return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14             meta = context.get("ffprobe_meta")
15             if not meta or not meta.get("streams"):
16                 return ValidationResult(
17                     validator_name=self.name,
18                     passed=False,
19                     message="Video metadata is missing. Run format_check validator first."
20                 )
21
22             stream = meta["streams"][0]
23             width = int(stream.get("width", 0))
24             height = int(stream.get("height", 0))
25
26             # Parse frame rate (e.g. "30/1" or "30000/1001")
27             fps_str = stream.get("r_frame_rate", "0/1")
28             try:
29                 num, den = map(int, fps_str.split("/"))
30                 fps = num / den if den != 0 else 0
31             except ValueError:
32                 fps = 0.0
33
34             details = {
35                 "width": width,
36                 "height": height,
37                 "fps": round(fps, 2)
38             }
39
40             # Check bounds from config
41             val_cfg = config.get("validation", {})
42             min_w = val_cfg.get("min_resolution_width", 1280)
43             min_h = val_cfg.get("min_resolution_height", 720)
44             min_fps = val_cfg.get("min_fps", 29.9)
45
46             if width < min_w or height < min_h:
```



```

47         return ValidationResult(
48             validator_name=self.name,
49             passed=False,
50             message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
51             details=details
52         )
53
54     if fps < min_fps:
55         return ValidationResult(
56             validator_name=self.name,
57             passed=False,
58             message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
59             details=details
60         )
61
62     return ValidationResult(
63         validator_name=self.name,
64         passed=True,
65         message="Video resolution and frame rate meet minimum standards.",
66         details=details
67     )
68

```

24. assistant (2026-06-22T15:06:19.587Z)

This further confirms it. The default validator chain requires ``ffprobe_meta`` to be present for ``ResolutionFPSValidator`` to even attempt its checks. If ``ffprobe_meta`` is missing, both ``format_check`` and ``resolution_fps_check`` fail ? ``failures`` non-empty ? early return at line 575, never reaching ``_maybe_dispatch_remote``.

Now, let me consider the one edge case the finding might still rely on: **could the validator registry be empty/customized** so that no validator populates ``ffprobe_meta`` yet validation "passes" (empty ``failures``)? If someone configures zero validators, ``val_results`` would be empty, ``failures`` empty, and ``ffprobe_meta`` never set ? ``fps_source == "unknown"``. Then the cloud path runs, and a pre-dispatch rejection would set ``segmentation_ran=True`` and fabricate `fps=30`.

But note: this same fabrication asymmetry would equally affect the **in-process success path** (which the finding acknowledges sets ``segmentation_ran=True`` legitimately). The finding's specific complaint is about **pre-dispatch rejections** ? the four ``_failed()`` cases. Let me assess each:

1. **Unresolvable input / local-only input** ? ``resolve_input_ref`` is called on ``req.video_path``. But ``acquire_local_input`` already ran at line 548. For a local-only input, the validators ran on the actual local file ? ``ffprobe_meta`` populated ? ``fps_source == "probed"``. So ``test_cloud_requires_a_bucket_input`` (local path) would have a probed fps if ffprobe works. In the test, validators are mocked to return ``(_pass(), {})`` ? empty context, so ``fps_source == "unknown"``. Let me check whether the report is even emitted/asserted in that test and what fps it produces.

Let me check whether the finding's scenario is actually reachable in practice vs only in mocked tests, and assess the real-world severity. Let me look at how ``emit_indexer_report`` handles the missing-fps and whether this is observably wrong.

25. user (2026-06-22T15:06:20.437Z)

```
<system-reminder>Warning: the file exists but is shorter than the provided offset (225). The
file has 224 lines.</system-reminder>
```

26. user (2026-06-22T15:06:23.643Z)

```
92: def _segmentation_stage(
```

```

189:     segmentation_ran: bool = True,
209:     if vm.get("fps_source") == "unknown" and segmentation_ran:
218:         segmenter_actual, config_default, was_reingest, segmentation_ran)
219:     _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran, run_signals)

```

27. user (2026-06-22T15:06:29.788Z)

```

92     def _segmentation_stage(
93         rec: "obsreport.RunRecorder",
94         play_segments: List[Dict[str, Any]],
95         candidates: List[Dict[str, Any]],
96         segmenter_requested: Optional[str],
97         segmenter_actual: Optional[str],
98         config_default: Optional[str],
99         was_reingest: bool,
100        ran: bool,
101    ) -> None:
102        with rec.stage("segmentation") as st:
103            if not ran:
104                st.status = "not_run"
105                seg_count = len(play_segments)
106                total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107                detector_tag = None
108                if play_segments:
109                    detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110                st.add_metric("candidate_windows", len(candidates))
111                st.add_metric("segment_count", seg_count, audience="both")
112                st.add_metric("total_play_s", total_play, unit="s")
113                # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114                # pick it, and it differs from config's default (the genuine surprise case).
115                # An explicit --segmenter override or a halted run is not a divergence.
116                matches_default = True
117                if ran and segmenter_actual and config_default:
118                    matches_default = (segmenter_actual == config_default)
119                st.details.update({
120                    "segmenter_requested": segmenter_requested,
121                    "segmenter_actual": segmenter_actual,
122                    "config_default": config_default,
123                    "segmenter_explicitly_requested": segmenter_requested is not None,
124                    "matches_config_default": matches_default,
125                    "was_reingest": was_reingest,
126                    "detector": detector_tag,
127                })
128                # Surface metadata trustworthiness at the data level (not just in a warning):
129                # the motion baseline fabricates per-segment metadata; everything else measures
130                it.
131                if ran and segmenter_actual:
132                    st.details["metadata_source"] = (
133                        "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
134                        else "measured (detector/AI)"
135                    )
136
137    def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                play_segments: List[Dict[str, Any]], ran: bool,
139                run_signals: Optional[Dict[str, Any]] = None) -> None:
140        """Only meaningful for AI-based segmenters. Records whether the provider key was
141        present ? the signal the silent_provider_noop rule keys on ? plus any in-process
142        run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
143        ai_based = segmenter_actual in _AI_SEGMENTERS
144        provider = (config.get("ai_provider") or "gemini")
145        key_env = f"{provider.upper()}_API_KEY"
146        with rec.stage("ai_handoff") as st:
147            if not ran or not ai_based:

```

```

148         st.status = "not_run" if not ran else "skipped"
149     st.details.update({
150         "ai_based": ai_based,
151         "provider": provider if ai_based else None,
152         "model": config.get("ai_model") if ai_based else None,
153         "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
154         # Chunks the provider refused as over its upload cap (no analysis ran for
155         # those time ranges). Detail name couples to the oversize_clip_skipped rule.
156         "oversize_clips_skipped": int((run_signals or
157     {}).get("oversize_clips_skipped", 0)) if ai_based else None,
158     })
159     if ai_based:
160         st.add_metric("confirmed_segments", len(play_segments))
161
162     def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
163                   video_meta: Dict[str, Any]) -> None:
164         total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
165         coverage = None
166         dur = video_meta.get("duration_s")
167         if dur:
168             coverage = round(100.0 * total / dur, 1)
169         notes = []
170         if not play_segments:
171             notes.append("No highlights were produced for this video.")
172         rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
173                           coverage_pct=coverage, notes=notes)
174
175     # --- top-level -----
176     def record_run(
177         *,
178         video_id: str,
179         video_path: Optional[str],
180         config: Dict[str, Any],

```

28. assistant (2026-06-22T15:06:41.252Z)

Now I have a complete understanding. Let me note an important nuance about the `\_segmentation\_stage`: it uses `ran` (segmentation_ran) to mark the stage `not\_run`. For the pre-dispatch rejection cases, the finding's complaint about `segmentation\_ran=True` would also make the segmentation stage NOT marked `not\_run` (it would show `seg\_count=0` as if it ran and produced nothing). So the asymmetry is real in principle for the segmentation stage too, not just fps.

Now let me weigh the verdict carefully. The finding makes two claims:

1. **Mechanically:** `if remote is not None: segmentation\_ran = True` fires for ALL dict returns including the four `\_failed()` pre-dispatch rejections? **TRUE**. This is verifiably correct from the code.
2. **The consequence (fps fabrication):** This requires `fps\_source == "unknown"` at report time. As I established, on the real cloud path, the input has been fetched to a local file and validated; if `FormatValidator` passed (required to reach `\_maybe\_dispatch\_remote`), `ffprobe\_meta` is set? `fps\_source == "probed"`. So the fps fabrication described would **NOT** trigger in normal operation.

Let me verify the in-process asymmetry claim the finding makes: "in-process equivalents (validation failure / unknown segmenter) leave segmentation_ran=False." Looking at the code: validation failure returns at line 575 with `segmentation\_ran=False`?; unknown segmenter returns at line 593 with `segmentation\_ran=False`?. So yes, the in-process error branches keep `segmentation\_ran=False`, while the cloud `\_failed()` branches return through line 607-608 setting it True. That asymmetry is real.

Let me confirm there's truly no `ffprobe`-independent path. The one realistic case where

`fps\_source` stays `"unknown"` despite passing validation: a config with the validators removed/customized. Let me check the default validator registration.

29. user (2026-06-22T15:06:43.619Z)

```
backend/pipeline/validators/base.py:38: def register(self, validator: VideoValidator):
backend/pipeline/validators/base.py:46:     ``context['ffprobe_meta']`` produced by
FormatValidator(). ``run_all``
backend/pipeline/validators/base.py:65:     """Runs all registered validators in sequence,
accumulating results."""
backend/pipeline/validators/format.py:8:class FormatValidator(VideoValidator):
backend/pipeline/validators/resolution_fps.py:4:class ResolutionFPSValidator(VideoValidator):
backend/pipeline/validators/__init__.py:2:from backend.pipeline.validators.format import
FormatValidator
backend/pipeline/validators/__init__.py:3:from backend.pipeline.validators.resolution_fps import
ResolutionFPSValidator
backend/pipeline/validators/__init__.py:10:registry.register(FormatValidator())
backend/pipeline/validators/__init__.py:11:registry.register(ResolutionFPSValidator())
backend/pipeline/validators/__init__.py:12:registry.register(PTSContinuityValidator())
backend/pipeline/validators/__init__.py:13:registry.register(SceneCutDensityValidator())
backend/pipeline/validators/__init__.py:14:registry.register(BlurValidator())
backend/pipeline/validators/__init__.py:15:registry.register(RelevanceValidator())
backend/worker/__main__.py:38:from backend.pipeline.validators.base import registry as
validator_registry
backend/worker/__main__.py:186:    val_results, val_context =
validator_registry.run_all_with_context(local_video, config)
```

30. assistant (2026-06-22T15:06:58.119Z)

Confirmed. `FormatValidator` is registered first by default and always populates `ffprobe\_meta` on success. The default production setup always probes fps and sets `fps\_source == "probed"` whenever validation passes (which is a precondition for reaching the cloud dispatch). So the fps fabrication described in the finding is not reachable in the default configuration.

Now my assessment of severity. Let me weigh it:

****The mechanical bug is real but the described impact is largely not.**** The core observation ? `segmentation\_ran=True` is set for pre-dispatch rejections, creating an asymmetry with the in-process error branches ? is verifiably true. However, the finding's headlined consequence (the "fake 30fps fallback" corrupting the report) requires `fps\_source == "unknown"`, which is ****not** reachable on the cloud path in any default/normal config**, because validation (which populates `ffprobe\_meta` ? `probed`) is a precondition for ever reaching `\_maybe\_dispatch\_remote`. The fps fabrication only manifests if validators are stripped/customized to pass without probing ? an exotic non-default config.

There IS a smaller real consequence the finding under-emphasizes that holds regardless of fps: for the four `\_failed()` pre-dispatch cases, `\_segmentation\_stage` will NOT mark the stage `not\_run` (because `ran=True`), so the report shows segmentation as having run with `segment\_count=0` rather than `not\_run` ? a mild observability inaccuracy, asymmetric with the in-process error branches. That's a genuine but minor report-fidelity defect.

Given:

- The mechanical asymmetry is real and confirmed.
- The dramatic "fake 30fps" corruption is essentially unreachable in default config (over-stated).
- The residual real impact (segmentation stage mislabeled as run-with-0-segments instead of not_run on cloud config/input errors) is a minor observability nit on an error path, default-OFF feature, behind a job worker.
- The fix is trivial and correct (gate on `remote.get("status") == "success"`).

This is a real-but-minor defect. The severity "minor" in the finding is appropriate, perhaps even generous given the headline impact is overstated. It holds up as a real (if small) correctness/observability asymmetry, so `is\_real=true`, severity `minor`.

31. user (2026-06-22T15:07:08.559Z)

Structured output provided successfully

32. assistant (2026-06-22T15:07:18.096Z)

Verified. The finding holds up as a real but minor defect.

Summary of my adversarial verification of `C:/Users/avidu/Projects/bhi-cloudrun/backend/main.py:606-610`:

****What's true (confirmed in code):****

- `if remote is not None: segmentation\_ran = True` does fire for ALL dict returns from `\_maybe\_dispatch\_remote`, including the four pre-dispatch `\_failed()` rejections where no segmentation ran.
- This is genuinely asymmetric with the in-process error branches (validation failure returns at main.py:575, unknown segmenter at main.py:593) which both keep `segmentation\_ran=False`.
- The real residual consequence: `backend/reporting/harvest.py:\_segmentation\_stage` (lines 100-104) won't mark the stage `not\_run` for these cloud config/input errors, instead reporting it as run-with-0-segments.

****What's overstated:****

- The headlined "fake 30fps fallback corrupting the report" (harvest.py:209-212) requires `fps\_source == "unknown"`, which is essentially unreachable on the cloud path in default config. Reaching `\_maybe\_dispatch\_remote` requires validation to pass, and the default-registered `FormatValidator` (validators/`\_\_init\_\_.py:10`) populates `ffprobe\_meta` on its success path (format.py:47) ? `fps\_source == "probed"` (harvest.py:54-55). The fps fabrication only manifests with a stripped/customized validator chain that passes without probing.

****Verdict:**** `is\_real=true`, severity `minor` (as proposed, arguably generous given the headline impact doesn't hold under default config). The proposed fix ? gating `segmentation\_ran` on `remote.get("status") == "success"` ? is correct and trivial.